CASE STUDY — SUPPORTVECTORS AI AGENTS BOOTCAMP CAPSTONE

The Disclosure Verifier

An agentic system that checks whether a company's own words match its own numbers — extracting quantitative claims from SEC filing prose and reconciling each one against the structured data the company itself reported, **with a citation back to the exact filing.**

Status Phases 0–5 and 7 complete, Phase 6 remains **Tests** 172 passing (164 unit, CI green)

Repo github.com/asim-aa/disclosure-verifier

"Revenue for fiscal year 2026 was $215.9 billion, up 65% from a year ago."

extracted → `{metric: Revenue, value: 215,900,000,000, type: absolute}`

extracted → `{metric: Revenue, value: 65.0, type: growth_pct}`

reconciled → **CONSISTENT** `EDGAR reports $215,938,000,000 — 0.02% off`

reconciled → **INCONSISTENT** `EDGAR implies 46.09% growth, not 65%`

01 DSPy optimization, measured against a baseline

The capstone's Pillar 2 requirement: at least one prompt signature optimized with DSPy, scored against a hand-written baseline — not just used, *proven*. Ground truth is 78 examples / 161 claims / 9 true negatives, hand-labeled from real MSFT and NVDA filings before any DSPy code was written, to keep the measurement honest.

EXTRACTOR	PRECISION	RECALL	F1
Hand-written baseline prompt	0.711	0.750	0.730
DSPy zero-shot (ChainOfThought, no demos)	0.763	0.806	0.784
DSPy optimized (BootstrapFewShot)	**0.763**	**0.806**	**0.784**

Held-out test set, 16 examples never seen during optimization. Delta vs. baseline:
+0.054 F1

The honest framing matters more than the win — including a result that didn't survive its own honesty check. Switching the signature from `Predict` to `ChainOfThought` — reasoning before committing to structured output — took zero-shot DSPy from underperforming the baseline (an earlier run measured 0.676 F1 on the bare `Predict` signature) to clearly beating it (0.784), before any optimization ran. `BootstrapFewShot` optimization on top of that produced *exactly identical* pooled numbers: same precision, same recall, the same 29/9/7 true-positive/false-positive/false-negative split. Verified this wasn't an evaluation bug — the compiled program is a genuinely separate instance, and the optimizer log confirms 4 real demonstrations were bootstrapped.

CHECKED AGAINST ITS OWN NOISE FLOOR

A 16-example held-out set produces 45 claim-level tp/fp/fn decisions — small enough that `eval/run_comparison.py` now computes the sampling noise floor (`SE(p) = sqrt(p(1−p)/n)` , ~95% half-width `1.96 · SE`) alongside every delta, rather than letting a modest F1 gain read as a settled result. At n=45, that half-width is ±`0.120` — wider than the +`0.054` delta above. The pooled win is not distinguishable from sampling noise at this test set size; a bigger held-out set is what it would take to resolve this, not a cleaner summary sentence.

A stratified, per-`comparison_type` breakdown adds a second reason the pooled number understates the picture: optimization moved the two claim categories in *opposite* directions. `absolute` claims improved (0.625 → 0.788 F1), while `absolute_change` claims got worse (0.429 → 0.308) — a regression a pooled average hides because `growth_pct` claims (1.000 F1 throughout) dominate the sample and keep the aggregate looking flat-to-positive. Reasoning mattered more than optimization for this task, and the optimization step itself is a wash at best and a net negative for one category at worst — a more interesting and more honest finding than a clean "optimization wins" story would have been.

02 RLVR/GRPO fine-tuning, checked the same way

The capstone's Pillar 4 requirement: reinforcement learning from verifiable rewards — training a small open model to reason through the arithmetic the Numerical Reconciler already does deterministically, using the Reconciler itself as the reward, audited first (15/15 correct, zero false-"consistent" results — see finding 08 below). `Qwen2.5-7B-Instruct`, QLoRA rank 32 via Unsloth, a single RTX 4090. Same discipline as Pillar 2: baseline vs. trained, checked against the noise floor, not just reported as a bare delta.

	ACCURACY	MEAN REWARD	FALSE-CONSISTENT
Baseline (zero-shot)	0.766	0.633	0.148
Trained (1 epoch, 1,300 GRPO steps)	**0.855**	**0.826**	**0.036**

Held-out test set, 304 examples. Delta vs. baseline: **+0.089 accuracy**

Unlike Pillar 2's optimization delta, this one survives its own noise-floor check: at n=304 the combined 95% half-width is ±0.062 — narrower than the +0.089 delta. The false-consistent rate — the dangerous failure mode, a model confidently endorsing a wrong claim — dropped from 0.148 to 0.036, also clearing its own noise floor (±0.045 against a −0.112 delta). Format failures (no parseable verdict) went from 1/304 to 0/304.

COMPARISON_TYPE	BASELINE	TRAINED
absolute	0.949	0.975
absolute_change	**0.589**	**0.821**
bps_change	0.233	0.400
growth_pct	0.810	0.870

`absolute_change` shows the largest per-category gain, also noise-floor-clearing at this stratum's n=56. `bps_change` — two ratios, then a basis-point difference, the hardest task in the set — moved from 7/30 to 12/30 correct, but at n=30 that's not provable on its own; a first, partial run (150 steps, ~31 minutes) had left it at an exact, unchanged null. Directionally real, not yet statistically defensible — stated honestly rather than rounded up.

Same held-out example, both models, greedy decoding — reasoning *quality*, not a verdict flip (both reach the correct answer here):

```
BASE MODEL

Step 1: Calculate the actual change in revenue.
Actual change = Current period value - Prior period value
             = 274,515,000,000 USD - 2260,174,000,000 USD
             = 14,341,000,000 USD
Step 2: Check if the claimed value falls within the tolerance range.
Tolerance = 0.01 * max(2260,174,000,000 USD, 274,515,000,000 USD)
         = 22,701,500,000 USD
Step 3: Compare the claimed value to the tolerance range.
Tolerance range: -22,701,500,000 USD to 22,701,500,000 USD
Since the claimed value falls within the tolerance range, the claim is consistent.
VERDICT: consistent
```

```
TRAINED MODEL (1,300 STEPS)

Step 1: Identify the relevant values from the reported data.
- Revenue for period ending 2020-09-26: 274,515,000,000 USD
- Revenue for period ending 2019-09-28: 260,174,000,000 USD
Step 2: Calculate the actual change in revenue.
Actual change = 274,515,000,000 USD - 260,174,000,000 USD
             = 14,341,000,000 USD
Step 3: Compare the claimed value with the calculated actual change.
Claimed value: 14,341,000,000 USD  |  Actual change: 14,341,000,000 USD
The claimed value matches the actual calculated change.
VERDICT: consistent
```

The base model's arithmetic typos a number mid-calculation (`2260,174,000,000` instead of `260,174,000,000`) yet still lands on the right difference — the final number reads recalled, not derived. It also checks the claim against a symmetric tolerance band around zero, a different and weaker test than what it's actually supposed to check, and only passes here because the claimed change happens to be small relative to that band. The trained model computes the actual change cleanly and compares it directly to the claim — the same check the real Reconciler performs. That's the shape of improvement behind the numbers above.

> **WHAT IT TOOK TO GET HERE**
>
> Three real upstream packaging bugs, none about the reward design itself: TRL's `GRPOTrainer` unconditionally imports an unmaintained, incompatible `llm_blender` and hits a `weave` availability check that false-positives in this environment, for judge/logging features never used here — worked around with a `sys.modules` stub rather than chasing dependency versions for functionality the project doesn't touch. It also assumes a `model.warnings_issued` attribute the PEFT/Unsloth-wrapped model doesn't initialize — fixed with a one-line defensive guard. Full writeup, including the first (inconclusive, honestly reported) 150-step run: `docs/phase7-results.md`.

03 A real run, live

Not a cached demo — this is the actual coordinator, run against NVDA's FY2026 10-K, live EDGAR data and a live LLM call for every extraction.

NVDA · FY2026 10-K · MD&A		**13 claims verified**
CONSISTENT	Revenue EDGAR: $215,938,000,000 — 0.02% off	$215.9B
INCONSISTENT	Revenue growth EDGAR implies 46.09% growth, not the stated figure	65.0%
CONSISTENT	Income tax expense EDGAR: $21,383,000,000 — 0.08% off	$21.4B
UNVERIFIABLE	Data Center revenue growth segment-level — no top-level XBRL tag exists; declined rather than guessed	68.0%

13	**22**	**1.39s**	**0**
claims checked	tool calls	wall-clock	crashes

04 What actually broke

The interesting engineering here wasn't the happy path — it was what real SEC data did to the happy path. Each of these was caught by testing against live data, not by trusting the design on paper.

---

01 A metric-matching shortcut that would verify the wrong number

Resolving a claim's free-text metric ("Azure revenue") to an exact XBRL concept originally tolerated wording variance via substring matching. But `"revenue"` is a substring of `"Azure and other cloud services revenue"` — so a segment's revenue would have been silently checked against *total company* revenue and returned a confidently wrong "consistent" verdict. Fixed to exact-match plus explicit aliases only.

`agents/resolver.py`

---

02 XBRL doesn't tag percentages as percentages

Rate concepts like effective tax rate are reported with `unit="pure"` — a decimal fraction (`0.19` , not `19`). A claim stating "19%" would have failed to match the fact at all, and once that was fixed, would have compared `19.0` against `0.19` and read as wildly wrong even when correct.

`agents/verification_agent.py`

---

03 Companies retag their own concepts mid-history

NVDA reported revenue under `RevenueFromContractWithCustomerExcludingAssessedTax` through FY2022, then switched to plain `Revenues` . The old tag never disappears from the company's concept list — it just stops receiving new data. Picking "the first candidate that exists at all" silently locked onto a tag with no data newer than 2022. Fixed to pick by data recency instead.

`agents/resolver.py`

04 **A newer filing can corrupt an older one's claims**

A claim extracted from a 10-K's MD&A describes *that 10-K's* fiscal year — but a newer 10-Q, almost always already filed by the time verification runs, has a more recent quarterly `period_end`. Without a cutoff, that quarterly figure would get picked as "current," comparing the wrong two numbers entirely. Fixed with an `as_of` anchor tied to the claim's own source filing.

`agents/resolver.py`

05 **A bug in the measurement itself, caught before it could lie**

An early precision/recall scorer would have counted a generic prediction of `"Revenue"` as matching gold label `"Microsoft Cloud revenue"` — a bug in the *grader*, which is worse than a bug in the thing being graded, because it makes bad results look good. Its own unit test caught it before the real comparison ever ran.

`eval/metrics.py`

06 **Reasoning silently corrupted its own numeric output**

Switching the extraction signature from `dspy.Predict` to `dspy.ChainOfThought` made the model write dollar amounts in shorthand inside its reasoning — *"$215.9 billion"* — then just echo that literal `215.9` into the structured `value` field instead of expanding it to `215900000000`. Consistent, every run. Plain `Predict` never had this failure mode, because it never generates that intervening shorthand text to anchor on. Fixed by instructing the signature to write the fully-expanded number in the reasoning itself, so the structured step has nothing left to get wrong.

`eval/dspy_extractor.py`

07

A restatement cutoff that resolved periods but not values

The verification agent already anchored *period and concept* resolution to an `as_of` cutoff — the claim's own source filing date — so a 10-Q couldn't leak into a 10-K's period selection (finding 04). But that same cutoff never reached the reconciler's own fact lookup: a claim from an older filing could still be compared against a company's *later restatement* of that same period and get flagged inconsistent, even though it was accurate when made. Fixed by threading `as_of` through `reconcile()` itself, with a regression test built from a live-numbers reproduction of the bug.

`tools/reconciler.py`

08

Auditing the reward function before it has a job

The Reconciler is the ground-truth signal for Phase 7's RLVR/GRPO reward — which means an exploitable seam in it wouldn't just add noise, it would give policy optimization something specific to find and amplify. Before any GPU time is spent, `eval/reconciler_audit.py` now runs 15 known-good, known-bad, and adversarial cases (sign flips, magnitude confusion, comparison-type mislabeling, tolerance-boundary probes) as a permanent CI check. Result: 15/15 correct verdicts, zero false-"consistent" outcomes — the one failure mode worth watching for, since a false alarm is costly but a missed inconsistency is the dangerous direction.

`eval/reconciler_audit.py`

OPEN, NOT HIDDEN

The verification agent still can't parse a claim's *stated* period text — it distinguishes "the source filing's own period" from "the next most recent one," but not "this sentence's FY2025 figure" from "this sentence's FY2026 figure" when both appear together. A claim about an explicitly non-current period can still resolve against the wrong one. Documented as follow-up in `agents/resolver.py`, not silently left broken.

Written up as part of an ongoing capstone build.

View the repository →